

Design rationale

KYC onboarding and compliance review desk

Denis Xhabraimi · September 2026

This is a record of why the system is built the way it is, including the options I rejected. It assumes you have not clicked the demo.

The system onboards a customer: it takes their details, sends them through an identity-verification vendor, screens the name against sanctions lists, scores the result, and either decides automatically or hands the case to a compliance officer at a review desk. Two services — a Next.js web application and a Python worker — share one PostgreSQL database and never call each other directly. The interesting problem is not the happy path. It is that the vendor drops webhooks, delivers them twice, and delivers them out of order, and the system has to stay correct anyway, and be able to prove afterwards that it was.

Everything below is a decision I would have to defend.

1. The principle underneath: fail closed

I applied one rule four times before I noticed I was doing it, so it is worth naming before the individual decisions: **prefer a visible failure to a silent degradation**.

A sanctions screen that runs against half a list returns fewer matches and looks like a clean result. A disclosure filter that shows an event type it does not recognise leaks. A shared secret that is unset and therefore lets everyone through is worse than one that locks everyone out. The failure that does not announce itself is the expensive one, because nobody goes looking for it.

So: the sanctions loader stamps “loaded” *last*, and the matcher refuses a snapshot without that stamp, so a half-finished load is unusable rather than quietly thin. The applicant-facing audit view is a whitelist that renders nothing for an event type it does not know. The worker’s HTTP endpoint refuses every request when its secret is unconfigured. The audit table rejects UPDATE and DELETE at the database level rather than by convention. None of those choices makes the system more capable. Each makes its failures louder.

2. The job queue is a Postgres table

This is the decision I would defend hardest, because it looks like the naive one.

The problem

When a verification result arrives by webhook I must do two things: record that it arrived, and schedule the work it implies. If they are not atomic, no ordering is correct. Enqueue first and the database write can fail, leaving a job that refers to a row which does not exist. Write first and the enqueue can fail, leaving a result nobody processes — an applicant waiting forever, no error anywhere.

What I chose, and what I rejected

The queue is an ordinary table in the same database as the business data, so enqueueing is part of the same transaction as the write that caused it. Either both happen or neither does. There is no window.

I rejected Redis and Celery for exactly this. Both are better queues in isolation — faster, with richer scheduling and visibility — and both are a *second* system, which means a distributed

transaction I would have to fake with an outbox pattern: write to Postgres, write a row saying “I still need to tell Redis”, drain that row into Redis. At which point I have built a database-backed queue anyway, plus Redis, and the database-backed queue is the one carrying correctness. I would rather have that without the dependency in front of it.

I rejected `pg-boss` for a different reason: it would have owned the schema. The database is shared by TypeScript and Python, and I did not want a JavaScript library’s migrations deciding the shape of a table the Python worker also writes to. Raw SQL in both services, with numbered migrations belonging to neither, keeps that boundary clean.

When this stops being a good idea

At high throughput. Every claim is a write and a row-level lock; the table accumulates dead tuples and leans on autovacuum. Somewhere in the low thousands of jobs per second this becomes the bottleneck and a purpose-built broker earns its operational cost. This system handles a few hundred applications a day — I am nowhere near it, and the honest version of the argument names the point at which I would change my mind.

3. FOR UPDATE SKIP LOCKED

With several workers running, two of them must never claim the same job. The claim is one statement:

```
with claimed as (  
  select id from jobs  
  where status = 'queued' and run_after <= now()  
  order by run_after, id  
  for update skip locked  
  limit 1  
)  
update jobs j  
  set status = 'running', attempts = j.attempts + 1,  
      locked_at = now(), locked_by = %s  
  from claimed c where j.id = c.id  
returning j.id, j.job_type, j.payload, j.attempts, j.max_attempts
```

`FOR UPDATE` takes a row lock. Without `SKIP LOCKED`, a second worker reaching the same row *blocks* until the first commits — every worker queued behind one row, which is the opposite of what a worker pool is for. `SKIP LOCKED` changes one thing: a locked row is treated as though it were not there, so the second worker takes the next one instead.

I did not want to assert this, so I measured the counterfactual. There is a deliberately broken claim function in the codebase — select, then update, with no lock — kept so the comparison is reproducible rather than rhetorical.

Five runs today, two workers against 200 jobs with no lock, gave 198, 183, 176, 192 and 198 duplicates. The third, exactly: **376 executions of 200 distinct jobs, 176 of them run twice**. That spread is wide and I would rather print it than narrow it — the count swings by twenty between runs on one machine, because this is a race, and how often two workers land on the same row is scheduling luck. The same five runs *with* the lock: 200 executions, 200 distinct jobs, zero duplicates, five times out of five.

A test asserts the broken version is *still broken*. That looks absurd until you consider how it fails: someone reads the function, sees a race, fixes it, and the comparison quietly becomes a claim about nothing.

Attempts increments at claim time, not on failure

A job that crashes the process hard enough — out of memory, a killed container — never reaches a failure handler, so a counter incremented on failure never moves. That job would be retried forever, killing a worker each time. Counting at claim time means every attempt is paid for whether or not anyone survived to report it. A poison pill exhausts its attempts and is parked instead of circulating.

4. The lifecycle lives in a database trigger

An application moves `started` → `submitted` → `checking` → `screening` → `decided`. The rule that matters is that nothing reaches `decided` except from `screening`: no application is ever decided without having been screened.

I had that rule in Python. It was correct, and it was in the wrong place. The same table is written by the TypeScript service and the Python worker, so a rule held in one of them is a rule the other can break — and the guarantee I actually wanted to make was about the *data*, not about one codebase’s discipline.

So it is a `BEFORE UPDATE` trigger raising `restrict_violation`:

```
if new.status is not distinct from old.status then
    return new; - not every update is a transition
end if;
if not exists (select 1 from application_transitions t
               where t.from_status = old.status
                  and t.to_status = new.status) then
    raise exception 'illegal transition: % -> %', old.status, new.status
    using errcode = 'restrict_violation';
end if;
```

The permitted transitions are rows in a table rather than a hard-coded list, so the rule is inspectable with a query and changing it is a migration — a reviewable commit — rather than an edit to a constant.

This is the same argument as the one for migrations belonging to neither service, which was a stated principle of the project from the start. I shipped the original version not following it. The Python table is still there as a mirror, with a comment saying the database is authoritative, because the worker benefits from knowing a transition will be refused before attempting it.

The cost: the rule now lives somewhere a developer reading the application code will not see it. That is a real downside and I think it is the right trade, because the alternative distributes a data guarantee across two languages and trusts both.

5. The audit log is append-only, enforced by the database

Every state change writes a row to `audit_events`. That table rejects `UPDATE`, `DELETE` and `TRUNCATE` through triggers.

“Append-only by convention” is not append-only. It is a code review standard, and the thing an auditor is worried about is precisely the case where someone decided the convention did not apply. Enforcing it in the database means the guarantee survives a developer with a good reason and a deadline.

It also binds me. The test suite cannot clear the audit table between runs, so tests that count audit rows have to record a high-water mark first and count only above it — and a demo script that resets the queue cannot reset the history. That friction is the guarantee being real rather than decorative.

What it does not give me: this is tamper-*evident* within the system, not tamper-proof. Anyone with superuser access can disable a trigger. Real immutability means WORM storage or an append-only log outside the database, and a hash chain linking each row to its predecessor would make undetected mid-history edits infeasible. I have not built either. I would rather say that than let “append-only” do more work in a conversation than it can carry.

6. Risk scoring is points, not machine learning

Each signal contributes a fixed number of points: a confirmed sanctions match 60, a politically-exposed-person match 30, residence in a jurisdiction subject to a call for action 40, a failed document check 40. Under 20 auto-approves, 20–79 goes to a human, 80 and over auto-rejects.

The argument is not that a model would be less accurate. It is that when a regulator, an ombudsman, or the customer asks why this person was refused, the answer has to be a sentence. “Sanctions match at 100% name similarity, downgraded to probable because the listed date of birth conflicts, plus a declined document check” is a sentence. “The model scored 0.83” is not, and neither is a SHAP plot.

Two consequences I chose deliberately. PEP matches cannot on their own reach the rejection threshold, because being a politically exposed person is not illegal and wholesale refusal — de-risking — is something regulators criticise; the law asks for enhanced due diligence, which means a human. And a sanctions match alone routes to review rather than rejection, because the screening finds *names*, and identity is a judgement a person makes.

Every scored application stores the ruleset version and the thresholds in force at the time, so a decision can be reconstructed after the rules have moved.

The cost: the weights are my judgement, not fitted to outcomes. A real firm would back-test them and have a model risk governance process. A points system is explainable and unvalidated, which is a genuinely different thing from being right.

7. Two audiences, two rules: the disclosure boundary

The applicant’s status page and the officer’s case view read the same audit table. They must not show the same thing.

The officer’s view renders the raw event payload, because an auditor needs the unvarnished record and a pretty-printer that dropped a field would defeat the purpose of keeping one. The applicant’s page is public — addressed by the application’s UUID, no login — and the same rows contain the risk score, the routing decision, and sentences of the form “*Sanctions match against X on OFAC SDN at 86% name similarity*”.

I originally shipped the applicant page rendering those rows directly. That was the most serious mistake in the project.

Three reasons an applicant is not shown that, none needing a citation to stand up. First, tipping-off: financial-crime regimes generally restrict telling a customer what screening has happened about them, and rather than work out where the line falls in each jurisdiction, the safe operating rule is that screening detail does not reach its subject — then the legal question never arises. Second, it teaches evasion: “matched at 86%, downgraded because the listed date of birth conflicts” tells someone exactly which field to change. Third, and most often overlooked, *most of these matches are wrong*: a name similarity is a finding about a name, not a person, and showing it attaches a sanctioned individual’s identity to an applicant who is, ordinarily, somebody else.

On the law specifically: in the UK the nearest provision is the Proceeds of Crime Act 2002 s.333A, but it is narrower than the rule I apply — it concerns disclosing that a suspicious activity report has been made or that an investigation is contemplated. I name it as the family of rule this sits

in, not as authority for it.

The implementation is a **whitelist that fails closed**. Each event type maps to a sentence written for the applicant; an action with no entry renders nothing at all. A new event type added later is invisible to applicants until somebody decides what that audience may be told, which is the safe direction for that decision to default in.

One subtlety worth knowing, because it is easy to undo: this works because the page is a React Server Component, so the raw rows never cross to the browser. Moving that list into a Client Component would serialise them into the payload again — leaking without a single line of rendering code changing. A comment would not survive that refactor, so there is a test that fetches the live page and asserts that “OFAC”, “name similarity”, the score and the raw payload keys appear nowhere in the response. It has a counterweight test asserting the page still renders the applicant’s own history, because a disclosure assertion is trivially satisfied by a blank page.

8. Sanctions provenance: what the hash does and does not prove

A decision has two inputs: the rules and the list. I versioned the rules early; the list carried no version at all, so the question an auditor actually asks — “*was this person screened against the list as it stood that day?*” — was not hard to answer, it was unanswerable.

Now every screening result points at a snapshot row recording the source, the publisher’s publication date, the record count, a SHA-256 of the file and when it was loaded. The chain runs decision → match → list version → digest.

What the hash proves: that two screening runs recording the same digest used byte-identical content, which makes “the same list” checkable rather than assumed.

What it does not prove. The digest is of the JSON file my downloader writes, not of the XML the US Treasury published, so it attests to my *parser’s output*, not to the publisher’s document. The file is gitignored and never committed, so a reader cannot recompute it from the repository. And it is recorded, not verified: nothing checks the digest on load, so a corrupted file would be recorded faithfully as corrupt. The right guard is cross-checking against OFAC’s own declared record count and the previous snapshot, which I have not built.

Staleness is also unsolved: the list is loaded at deployment and nothing refreshes it. Recording the publication date makes that *visible* — an old date next to a recent decision is readable evidence — which is better than invisible, but not the same as fixed.

9. Two-stage matching, and one principle about pre-filters

Fuzzy name matching against 19,393 OFAC entries (44,017 searchable spellings once aliases are included) originally happened in memory: 86 MB resident, scanned per applicant. **Every one of those entries is loaded and searched**, vessels and companies as well as people. A subset of 7,429 — the person-shaped names that also carry a date of birth — appears only in the seed script, which draws synthetic applicant names from it so that some applicants deliberately resemble somebody listed. It narrows who gets *invented*, never what anybody is screened *against*. That is fine for a process impossible for one that must start, answer and exit.

So the list moved into Postgres and the search became two stages. A trigram GIN index over individual name *tokens* narrows the list to a shortlist — a median of 224 entries and a mean of 424, roughly 2% of the list — and then shortlist with exactly the code that ran before.

I indexed tokens rather than whole names, and that was the real decision. Across the 131 matches the scorer accepts for 400 applicants, whole-string trigram similarity falls as low as 0.194; even a pair as recognisable as **ahmed hassan** against **ahmad hasan** scores only 0.471. The reason is that transliteration drift lives inside words, and one extra middle name wrecks a whole-string

comparison while leaving every other token identical. The best-matching *token* in those same 131 pairs never falls below 0.357, and its median is 1.000 — the name part that matches usually matches exactly. A whole-string pre-filter at 0.25 would drop one of the 131; the per-token shortlist at the same 0.25 drops none.

The principle

My first version of the shortlist required two trigram-matched tokens, mirroring the Python rule that two name parts must correspond. It looked right and was wrong, because it applied that rule using a *different similarity metric* from the one the scorer uses, and the two disagree badly on short tokens and substrings: `chen/jicheng` scores 72.7 on the scorer’s metric (accepted) and 0.182 on trigram similarity (rejected).

Thirteen applicants out of 400 lost a real match. Every one was a miss, never a false positive — which is the dangerous direction.

A cheap pre-filter must only ever widen. The moment it reimplements the expensive filter’s judgement with a cheaper metric, it can disagree with it, and every disagreement is a match nobody sees. The shortlist now asks the weakest useful question — does this entry share any plausible name part? — and leaves every actual decision to the scorer.

The threshold was then set by measurement rather than taste: 0.35 lost 13 applicants’ matches, 0.30 lost one, 0.25 loses none. I chose 0.25 because recall is worth more than shortlist size here — a shortlist missing the one true match costs a sanctioned person an account. That choice is not free, and the cost lands on the tail.

What it costs

Faster for almost everyone, slower for the worst case. Over 300 applicants on one machine, by shortlist size — 10–49 entries: 190 ms in memory against 13 ms two-stage; 50–199: 204 against 27; 200–999: 212 against 65; 1,000 and over: 228 against 177. Worst observed, 457 ms against 623.

The in-memory scan is flat by construction, 190 to 228 ms, because it compares against all 19,393 entries whatever the name. Two-stage spans 13 to 623 ms, a 48× spread, because its cost is set by how many entries the pre-filter admits: a common surname on a list full of similar spellings produced a 4,608-entry shortlist and a slower answer than scanning everything. So — typically three to eight times faster, occasionally worse, and *less predictable*. I took that trade because the alternative was never a fast in-memory scan, it was a function that could not hold the list at all. But a latency budget here would have to be written against the tail, and the tail is the number that moves.

10. Differential testing, and the bug that was already shipping

Changing how candidates are *found* must not change who gets flagged. A silent shift in that is the worst outcome available: nobody sees it, the tests stay green, and the only evidence is a different set of people being refused.

So before trusting the new matcher I wrote an equivalence test that runs both over 400 seeded applicants and compares the matched entities, the strengths, the matched spelling, the date-of-birth conflict flag, the resulting score and the routing. It found three real bugs, all of which would have shipped.

One of them was not mine. An applicant matched three OFAC entities at *exactly* 100.00 — two with no date-of-birth conflict, one with. The scorer takes the strongest hit of each type, which with `max()` means the first of several equals. So whichever entity happened to be listed first decided whether the match was downgraded: the same applicant, against the same list, scored 60 or 75 depending on dictionary iteration order.

That was already in production behaviour before the migration. Running two implementations side by side is what made it visible; neither implementation was wrong on its own, and no single-implementation test would have caught it, because there was nothing to compare against. Both now sort by score and then by entity id, which makes the tie-break a decision rather than an accident.

I include this because “why should I trust your judgement” is a fair question and the honest answer is not that I do not make mistakes. It is that I try to build the thing that would tell me.

11. When one service may call another

The two services communicate only through the database. That rule exists to avoid an internal API — one service asking the other to do something and waiting to hear how it went, which drags CORS, shared auth, retries and a second failure mode into a design that had none.

The deployed version bends it: after enqueueing work, the web application sends an HTTP request to the worker so an applicant sees a decision in seconds rather than waiting for the next scheduled run. I let myself do that only because it passes a specific test:

- the work is already durable — committed to the queue — before the call
- the call carries no payload and expects no answer
- it is not awaited, and cannot fail the request that triggered it
- a scheduled path does the same work if it never arrives

Which reduces to one sentence: **delete the call and the system is still correct, only slower.** The queue still carries all the meaning; the HTTP request carries only timing.

The test then proved itself unplanned: the call silently did nothing for a period after deployment, and nothing broke. Applications were still decided by the scheduled path, minutes later instead of seconds — a latency regression rather than a correctness one, exactly as the design predicted.

12. The worker as an on-demand function

Locally the worker is a long-lived loop: claim, run, sleep, repeat. That is the right shape, and it is what `docker compose up` gives you. It is also a process that must run somewhere permanently, and permanently costs money. I am not paying a monthly bill to host a portfolio demo.

So production has a second entry point: an HTTP endpoint that claims a *bounded* batch — five jobs — runs them and returns. Bounded because a function has a wall-clock limit, and a loop running until the queue emptied would be killed mid-job on every invocation once a backlog existed, making no progress while looking busy. It is called repeatedly instead: by the web app when work is enqueued, and every five minutes by a scheduler that also runs the recurring jobs.

The queue’s guarantees are untouched, which is the part I care about. Both entry points import the same function to execute a job, so the transaction boundary and the retry accounting cannot diverge; jobs are still claimed one at a time with the same statement, and the 500-job no-double-processing proof runs against both. Same logic, two runtimes — rather than pretending the deployed thing is the same thing.

The cost: an applicant arriving when the database has been idle waits about 2.4 seconds for the first page that queries it, then roughly 0.3 for every page after. The five-minute schedule keeps the database awake, so this is paid only if the schedule has stopped.

Synthetic data throughout; no real person’s data has been in this system. Every figure was measured, not estimated; where one moves, I say so.